

<< 神经网络与深度学习 >>

Neural Networks and Deep Learning

§1 绪论

AI: 机器的行为 $\xrightarrow{\text{像}}$ 人的智能行为.

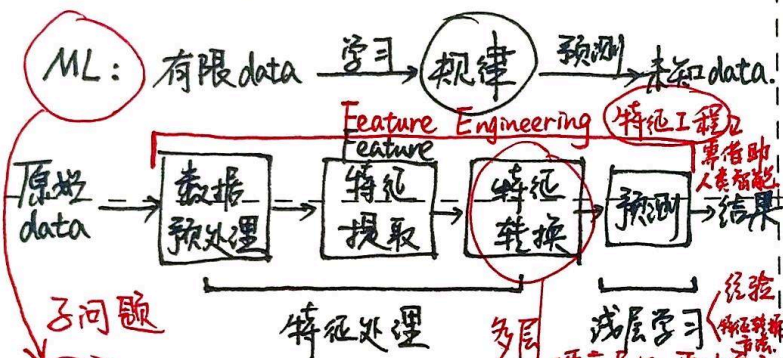
领域: 感知、学习、认知.

范畴: 机器感知、学习、语言、记忆、决策.
(视觉、语音) (PR, ML, RL) (NLP) (表示) (规划、控制)

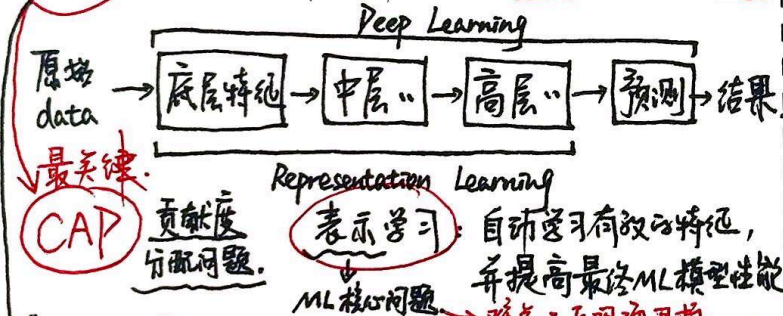
1950 Alan Turing 阿兰·图灵 — 图灵测试.

1956 达特茅斯会议 — AI定义.

1956 $\longrightarrow$ 1970s $\longrightarrow$ 1980 $\longrightarrow$ 至今
推理期 (逻辑+规则) 知识期 (专家系统) 学习期 (ML, DL)



DL: 从数据中自动学习到有效的特征表示.



语义鸿沟: 底层特征 $\leftarrow$ different $\rightarrow$ 高层语义信息.

局部表示 (离散/符号...) vs. 分布式表示 (distributed).

eg. $[0, 1, 0, 0]^T$ vs. $[1, 0.75, 0]$.

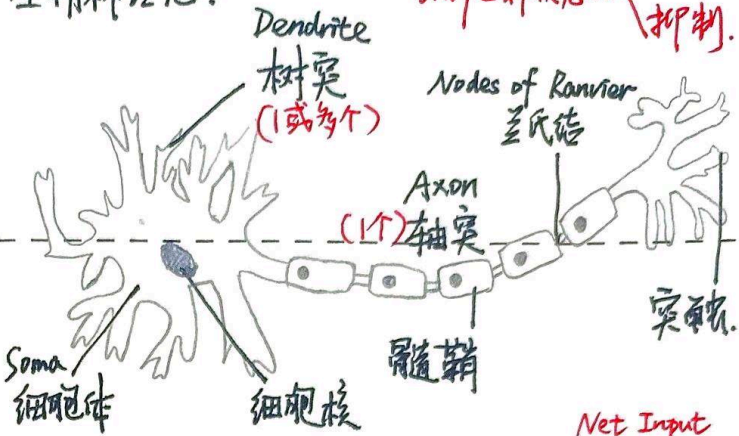
End-to-end Learning. 端到端学习. 直接优化总体目标.

大部分采用NN的DL都是.

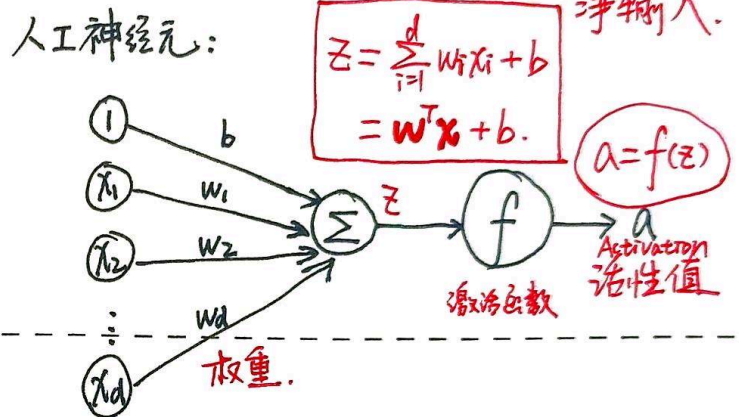
人工神经网络 (ANN): 大量神经元及其有向连接.

考虑3方面: 神经元的激活规则. 网络的拓扑结构. 学习算法.

生物神经元:

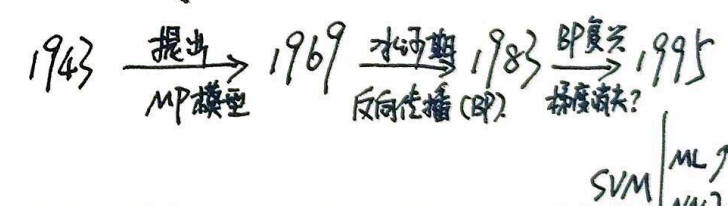


人工神经元:



记忆 (人脑) $\left\{ \begin{array}{l} \text{短期记忆} \\ \text{长期记忆} \end{array} \right. \rightarrow$ 凝固作用.

NN发展史:



Feedforward §4 前馈神经网络 (FNN).

激活函数的性质

- 连续、可导、非线性
- 尽可能简单
- 计算效率↑
- 值域在合适区间
- 整体呈上升趋势(局部可下降)

常用激活函数:



Logistic: $\sigma(x) = \frac{1}{1+e^{-x}}$. $\sigma'(x) = \sigma(x) \cdot [1-\sigma(x)]$

Tanh: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. $f'(x) = 1 - f^2(x)$

零中心化

$\tanh(x) = 2\sigma(2x) - 1$. (转换)

非...? 使后一层神经元 input 发生 偏置偏移, 并进一步使得梯度下降的收敛速度变慢!

改进? ① 归一化到零中心 or ② 加偏置项: $\sigma(x) + b$. b : 可学习参数

修正线性单元

ReLU(x) = max(0, x).

带泄露的 ReLU

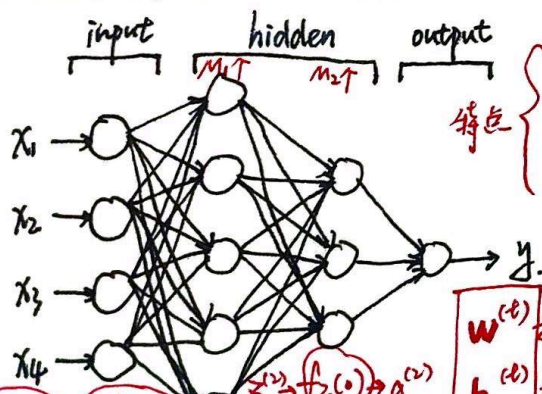
Leaky ReLU(x) = $\begin{cases} x, & x > 0 \\ \gamma x, & x \leq 0 \end{cases}$

指数线性单元

ELU(x) = $\begin{cases} x, & x > 0 \\ \gamma(e^x - 1), & x \leq 0 \end{cases}$

Softplus(x) = $\log(1 + e^x)$.

FNN FCNN MLP 前馈神经网络 / 全连接 NN / 多层感知机:



特点: 层内无连接. 相邻层两两连接. 无反馈, 单向传播 (有向无环图).

$w^{(l)}$ 指 $l-1 \rightarrow l$ 权重矩阵. $b^{(l)}$ 指 $l-1 \rightarrow l$ 偏置. (可学习参数)

层数 L 个数 M_L . $z^{(1)} \rightarrow f(\cdot) \rightarrow a^{(1)}$. 人为定义一起是数. 活性值(动态)

★ FNN 传播公式:

$z^{(l)} = W^{(l)} \cdot \underbrace{a^{(l-1)}}_{\text{上层 output}} + b^{(l)}$
 $a^{(l)} = f_l(z^{(l)})$

前馈计算: 复合函数 $\phi(x; W, b)$. input. 可学习参数.

$x = a^{(0)} \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow z^{(2)} \rightarrow \dots \rightarrow a^{(L)} = \phi$

Gradient 梯度计算 { 链式法则 反向传播 (BP).

自动微分 (AD). < US. 数值 vs. 符号 (介于二者之间!)

BP 推导 (不作要求).

误差项 $\delta^{(l)} \triangleq f'_l(z^{(l)}) \odot (W^{(l+1)})^T \delta^{(l+1)}$

结论: $\delta^{(l)}$ 可由 $\delta^{(l+1)}$ 得到!

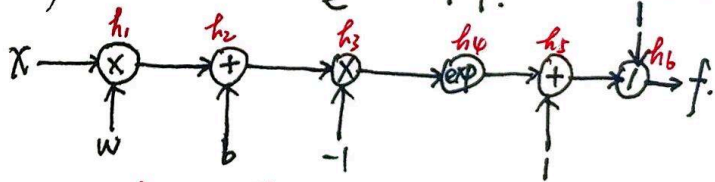
即 $\delta^{(l)}$ 为 $\delta^{(l+1)}$ 的权重和 $\odot$ 激活函数梯度. 元素相乘.

补充: 误差项反应贡献程度, 较好地解决了 CAP!

流程: 选取样本 $\rightarrow$ 前馈 $\rightarrow$ 反向计算 $\delta^{(l)}$ $\rightarrow$ repeat... until 在 val 的 错误率不再下降. 更新 w, b

AD: 分解基本操作 $\rightarrow$ 链式法则. (采用图的方式计算)

例如: $f(x; w, b) = \frac{1}{1 + e^{-(wx+b)}}$



$a \rightarrow b$ 求的是 $\frac{\partial a}{\partial b}$ (尖端对尾端的偏导!)

模式 { 前向模式 反向模式 $\rightarrow$ 和 BP 计算梯度方式相同!

计算图 { 静态...: 可优化, 并行能力强 / 灵活性差 动态...: 不易, 难以... / ...高!

优化问题

(1) 非凸优化问题.

存在局部最优而非全局最优, 影响迭代!

(2) 梯度消失 / 梯度弥散.

Sigmoid 饱和区导数 $\rightarrow 0$

误差经过每一层传递不断衰减!!

一种方法: 使用导数较大的 f , 如 ReLU.

怎样算“好”? $f(x) \rightarrow 1$.

导数 $= 1$

优化的需求?
 { 计算资源大
 数据多
 算法效率高 (收敛快) }

“搞 AI 就得有数据、高算力、好算法!”

补充: FNN 中的参数学习.

采用交叉熵损失函数, 对于样本 (x, y) :

$$\mathcal{L}(y, \hat{y}) = -y^T \log \hat{y}.$$

其中 $y \in \{0, 1\}^C$ 为 Label y 对应 one-hot 向量表示.

数据集 D 的结构化风险函数:

$$\mathcal{R}(W, b) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, \hat{y}^{(n)}) + \frac{\lambda}{2} \|W\|_F^2$$

正则化项

使用梯度下降进行学习:

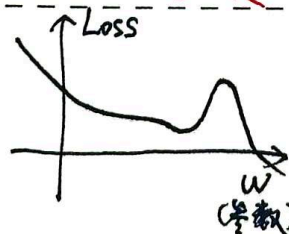
① 初始化 W .

② repeat:

计算 $\frac{\partial \mathcal{R}}{\partial W}$; 更新 $W \leftarrow W - \eta \frac{\partial \mathcal{R}}{\partial W}$.

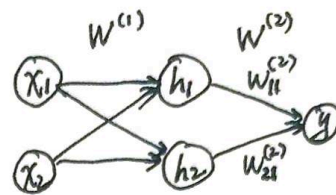
缺点? 反复求偏导, 低效!

故常用 BP $\rightarrow$ 更高效计算 gradient.



[习题 4-2] 设计 FNN 解决异或 (XOR),
 2 隐藏 + 1 输出神经元. ReLU.

解: 即要对于 $X = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ 分别得到 $y = 0, 1, 1, 0$.



Q: ReLU 何用?

A: 丢弃负值!!

取 $W^{(1)} = \begin{bmatrix} 1 & 1 \end{bmatrix}$, $b^{(1)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$, $X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

此时 $H^{(1)} = \text{ReLU}(W^{(1)T} X + b^{(1)}) = \begin{bmatrix} x_1 + x_2 \\ x_1 + x_2 - 1 \end{bmatrix}$

再取 $W^{(2)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$, $b^{(2)} = 0$, ($h_1, h_2 \geq 0$)

此时 $y = \text{ReLU}(W^{(2)T} H^{(1)} + b^{(2)}) = \text{ReLU}(h_1 - 2h_2)$.

(注: 以上步骤省略了 $H^{(2)}$, y 步骤的 ReLU.)

验证: (1) $\begin{bmatrix} 0 \\ 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 \\ -1 \end{bmatrix} \rightarrow 0$
 $\text{若 } x < 0, \text{ReLU}(x) = 0.$

(2) $\begin{bmatrix} 0 \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ -1 \end{bmatrix} \rightarrow 1$

(3) $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ -1 \end{bmatrix} \rightarrow 1$

(4) $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} 2 \\ -1 \end{bmatrix} \rightarrow 0$

[习题 4-3] 举例说明“死亡 ReLU”, 提出解决...

参数不恰当更新. 比如乘了一个负数?

$\downarrow$

某个 ReLU 神经元在所有训练数据上无法激活.

$\downarrow$

这个神经元自身参数的梯度永远为 0!

e.g. 上一题的情形 (1).

③ 正则化的束

④ 其他方式训练 (如 Adam)

解决方法 ① 改用 Leaky ReLU / ELU ...

② 优化初始学习率.

小一点!!

避免太大, 突然变负!!

强判变 0, Game Over!!

Convolutional

§5 卷积神经网络 (CNN)

FCNN 缺点

- W 的参数太多 $\rightarrow$ 效率低
- 易过拟合!!
- 局部不变性特征

对于自然图像中的物体, 都有...
 平移/缩放/旋转不变 (不影响语义信息)
 FCNN 很难提取这些特征! (需配合数据增强)

CNN $\subseteq$ FNN (CNN 是一种 FNN)

受生物学的 感受野 的机制而提出!

CNN 由以下层组成:

- 卷积层
- 汇聚层
- 全连接层

交叉堆叠而成!

3个特性:

- 局部连接
- 权重共享
- 池化/汇聚 (时/空间上的次采样)

$\rightarrow$ 使得 CNN 具有一定程度的 平移、缩放、旋转不变性

一维卷积 $\rightarrow$ 信号处理, 计算延迟累积

二维卷积 $\rightarrow$ 图像处理, 结果称为 特征映射 (Feature Map)

eg. $Y = W * X$. ($*$ = 二维卷积)

$W \rightarrow$ "滑动窗口". (卷积核/滤波器)

根据定义, 需卷积核翻转.

不过一般用互相关 $\otimes$ 替代卷积.

$(W \otimes X)_{i,j} = \sum_u \sum_v W_{u,v} X_{i-u, j-v}$ (CP12~113)

卷积的变种: 引入步长 (S), 零填充 (P).

对 M 个神经元, 卷积大小为 k, 则该卷积层神经元数量:

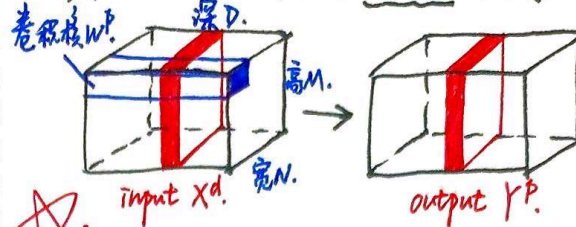
$M^* = (M + 2P - k) / S + 1$

窄卷积: $S=1, P=0 \Rightarrow M^* = M - k + 1$

宽卷积: $S=1, P=k-1 \Rightarrow M^* = M + k - 1$

半宽卷积: $S=1, P=(k-1)/2 \Rightarrow M^* = M$

典型的卷积层为 3 维结构.



(卷积核 W^P 的深度与输入一致 (D))

表达式: $Z^P = W^P \otimes X + b^P$
 $= \sum_{d=1}^D W^{P,d} \otimes X^d + b^P$

$Y^P = f(Z^P)$ $\rightarrow$ 三维卷积核

$W \in \mathbb{R}^{u \times v \times D}$

参数总量 (空间复杂度): $P * D * (u * v) + P$ (偏置项)

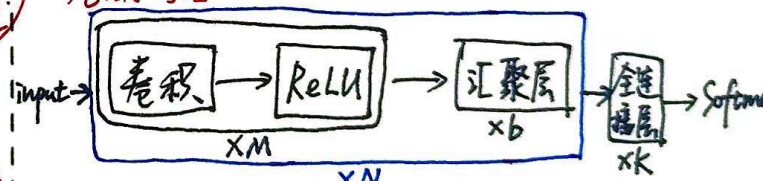
★ 看懂动图! (总复习 PPT, Page 52)

汇聚层 (子采样层): 进行特征选择, 降低特征数量, 从而减少参数数量.

Pooling $\rightarrow$ 对每个区域下采样, 作为这个区域的概括.

- (1) 最大汇聚 $\rightarrow$ 图见 P10. 要会计算!
- (2) 平均汇聚.

卷积网络:



转置卷积: 低维特征 $\xrightarrow{\text{映射}}$ 高维特征
 $X = \text{rot}(180(W)) \otimes Z$ (反卷积)

(或用宽卷积, $S=1, P=k-1 \Rightarrow M^* = M + k - 1$)

如何增加输出单元的 感受野?

- 增加卷积核大小. $k \uparrow$
- 增加层数. $L \uparrow$
- 在卷积之前汇聚.

Q: 不改变参数数量行不行? $(D-1) \uparrow$

空洞卷积: 给卷积核插入 "空洞", 来变相增加其大小.

D: 膨胀率. ($D=1$ 为普通卷积核)

几种典型的 CNN:

(1) LeNet-5. 支票手写数字识别. 7层.

Conv. → Pool. → Conv. → Pool. → Conv. → FC → output

(2) Alex Net. (5+3+3).

(GPU并行) + ReLU + Dropout + 数据增强
防止过拟合 提高准确率.

(3) Inception 网络. (多个 Inception 模块 + 少量 Pool.)



堆叠: 经过串联恢复到 input 通道大小.

(4) ResNet (残差网络). — 多个残差单元串联.

Why? 层数堆叠 添加恒等映射 > 都没马用 ~
反而附近扰动更的学习!!

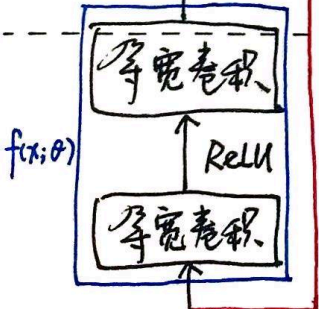
给非线性的卷积层增加直连边. (残差连接).

$$h(x) = x + \underbrace{f(x; \theta)}_{\text{残差函数}}$$

恒等函数

优势: $\frac{\partial h(x)}{\partial x} = 1 + \frac{\partial f(x; \theta)}{\partial x}$

ReLU
x (本身)
不易导致梯度消失!!
可以“记得很深”
且没有带来额外参数和计算开销.



(残差单元).

[习题 5-3] 分析 1×1 卷积核的作用.

① 降维: Inception 网络的 Shortcut 连接, 1×1 卷积用于减少特征映射的深度.

② 升维: ResNet 中, $1 \times 1 \times 256$ 卷积核将 output 64 维升为 256 维, 且只需最少参数 (1×256).

③ 增加网络的非线性特征, 提高拟合能力.

④ 降低时间复杂度. (见下题).

[习题 5-4] 求二者时 (空) 复杂度. 只考虑卷积核的.

$$\begin{aligned} & 3 \times 3 \text{ Conv} \rightarrow 100 \times 100 \times 256 \quad (1) \\ & 100 \times 100 \times 256 \xrightarrow{1 \times 1 \times P_1} 100 \times 100 \times 64 \xrightarrow{3 \times 3 \times P_2} 100 \times 100 \times 256 \quad (2) \end{aligned}$$

解: (1) 时间: $(100 \times 100 \times 256) \times (3 \times 3 \times 256)$
空间: $256 (3 \times 3 \times 256 + 1)$ 偏置.

(2) 时间: $(100 \times 100 \times 64) \times (1 \times 1 \times 256) + (100 \times 100 \times 256) \times (3 \times 3 \times 64)$

$P = \text{输入的 } D \quad (P_1 = D_0, P_2 = D_1)$

空间: $1 \times 1 \times 256 \times 64 + 3 \times 3 \times 64 \times 256 + \frac{P_1}{P_2} \times \frac{P_2}{P_1} \times 256 \times 64$
 $u_1 \cdot v_1 \cdot P_1 \cdot D_1 + u_2 \cdot v_2 \cdot P_2 \cdot D_2$ 可省略

[习题 5-5] input 3×3 , 卷积核 2×2 , 重写为仿射变换.

$$X = \begin{bmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{bmatrix}, \quad W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

仿射变换: $z = W \otimes X = CX$
($C \in \mathbb{R}^{4 \times 9}$).

执行滑动窗口可得方程:

$$\begin{cases} x_1 w_{11} + x_2 w_{12} + x_4 w_{21} + x_5 w_{22} = 0. \\ x_2 w_{11} + x_3 w_{12} + x_5 w_{21} + x_6 w_{22} = 0. \\ x_4 \sim + x_5 \sim + x_7 \sim + x_8 \sim = 0. \\ x_5 \sim + x_6 \sim + x_8 \sim + x_9 \sim = 0. \end{cases}$$

即 $z = CX = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ w_{11} & w_{12} & 0 & w_{21} & w_{22} & 0 & 0 & 0 & 0 \\ 0 & w_{11} & w_{12} & 0 & w_{21} & w_{22} & 0 & 0 & 0 \\ 0 & 0 & 0 & w_{11} & w_{12} & 0 & w_{21} & w_{22} & 0 \\ 0 & 0 & 0 & 0 & w_{11} & w_{12} & 0 & w_{21} & w_{22} \end{bmatrix} \cdot X$

Recurrent

§6 循环神经网络 (RNN)

Time Delay

延时神经网络 (TDNN).

短期记忆!

—— 给每个隐层都加一个延时器。

循环神经网络 (RNN).

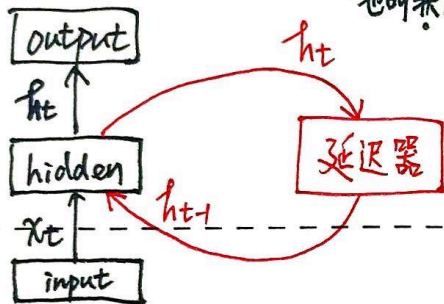
使用带自反馈的神经元,

可处理任意长度的时序数据。

随时间变化, 分布/类型也变化。

$$h_t = f(h_{t-1}, x_t)$$

上一时刻活性值 当前样本。
也叫状态 (State)。



FNN $\xrightarrow{\text{可模拟}}$ 任何连续函数。

RNN $\xrightarrow{\dots}$ 任何程序!

简单循环网络 (SRN). 只有1个隐层!

—— 增加了从隐层到隐层的反馈连接。

$$\begin{cases} z_t = U h_{t-1} + W x_t + b. \\ h_t = f(z_t). \end{cases}$$

按时间展开 $\rightarrow$ 可见 RNN 在时间维度上权值共享。
(图见 P16)

应用到 ML

- 序列 $\rightarrow$ 类别 ^{e.g. 情感分类}
- 同步的序列 $\rightarrow$ 序列 ^{e.g. 中文分词}
- 异步的序列 $\rightarrow$ 序列 ^{e.g. 机器翻译}

参数学习 — 随时间反向传播 (BPTT).

将 RNN 看作展开的 FNN, “每层” $\rightarrow$ “每个时刻”,

于是可用 FNN 的 BP 算法计算参数梯度!

所有层的参数共享! $\Rightarrow$ 真实梯度 $= \sum$ 各层梯度。

$$\begin{aligned} \delta_{t,k} &= \frac{\partial L}{\partial z_k} = \text{diag}(f'(z_k)) U^T \delta_{t,k+1} \\ &= \prod_{t=k}^{t-1} (\text{diag}(f'(z_k)) U^T) \delta_{t,t} \end{aligned}$$

(近似)

令 $\gamma \equiv \|\text{diag}(f'(z_k)) U^T\|$, 则 $\delta_{t,k} \approx \gamma^{t-k} \delta_{t,t}$.

• 若 $\gamma > 1$, $t-k \rightarrow \infty$, $\gamma^{t-k} \rightarrow \infty$, ① 梯度爆炸!

• 若 $\gamma < 1$, $t-k \rightarrow \infty$, $\gamma^{t-k} \rightarrow 0$, ② 梯度消失!

此即 长程依赖问题 (Long-Term Dependencies).

如何改进? $\rightarrow$ RNN 在时间维度上非常深!

• 对于①:

a. 权重衰减: 给参数 $\rightarrow$ P187. §7-7 增加 L2 范数的正则项, 使 $\gamma \leq 1$

b. 梯度截断: (启发式) 梯度的模大于阈值就截断。

• 对于②: 改变模型! 目标: 使 $\gamma \rightarrow 1$!

原始: $h_t = f(U h_{t-1} + W x_t + b)$. 循环递归构造。

改进: $h_t = h_{t-1} + g(x_t; \theta)$. 参数 θ .
令 $u = I$ (单位阵) 线性. 非线性函数 g .
但降低了表示能力!

再改进: $h_t = h_{t-1} + g(x_t, h_{t-1}; \theta)$.

仍存在 2 个问题:

<1> 梯度爆炸.

令 $z_k = U h_{k-1} + W x_k + b$ 作为 k 时刻 $g(\cdot)$ 输入,

$g(\cdot)$ 通过 ReLU / Logistic 为正, 那么 $h_t > h_{t-1}$.

说白了就是 (h_t) 一直变大变大变大变大!

<2> 记忆容量 (Memory Capacity) 问题.

若 $g(\cdot)$ 为 logistic, $t \rightarrow \infty$, h_t 饱和.

丢失的信息会越来越多!!

怎么办? 必须“忘记”一些东西!

$\rightarrow$ 门控!

基于门控的循环神经网络

(1) Long Short-Term Memory Network
长短期记忆网络 (LSTM).

包括有选择地加入新的信息。

门控机制 (控制信息的累积速度) 有选择地遗忘之前累积的信息。

$$\begin{cases} \text{内部状态 } C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\ \text{外部状态 } h_t = o_t \odot \tanh(C_t) \end{cases}$$

C_t 专用于线性的循环信息传递, 同时非线性的输出信息给随层的外部状态 h_t . C_t 记录了到当前时刻为止的历史信息.

$f_t, i_t, o_t \in [0, 1]^D$.
↓
控制 C_t 有多少遗忘.
↓
控制 C_t 有多少要保存.
↓
控制 C_{t-1} 需遗忘多少信息.

(图见 P146)

$$\begin{cases} \tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \\ i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \\ f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \end{cases}$$

形式一样, 记一个!

步骤: 计算 $\tilde{C}_t$ 和三个门 → 更新 C_t → 输出 h_t .

(2) Gated Recurrent Unit
门控循环单元 (GRU) 网络. $\hookrightarrow$ 比 LSTM 简单!
不引入额外的记忆单元, GRU 引入了更新门 z_t .

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot g(x_t, h_{t-1}; \theta)$$

其中 $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$.

LSTM 的 i_t, f_t 互补, 冗余! z_t 直接控制输入/遗忘平衡!

上式的 $g(\cdot)$ 即为 $\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$.

其中重置门 $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$,
用于控制候选状态 $\tilde{h}_t$ 是否依赖 h_{t-1} .

综上, $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$.

(图见 P149)

[习题 6-1] 分析 TDNN, CNN, RNN 的异同点.

同: ① 提取局部特征.
② 权重共享. $\star$ (延时层, 卷积核, 循环单元)

异: ① TDNN, RNN 时间上权重共享.
CNN 空间上权重共享.
② TDNN 依赖于上一层最近 k 个时刻活性值.
RNN 依赖于之前所有时刻活性值.
而 CNN 不存在这种相关性.

[习题 6-3] 使用 $h_t = h_{t-1} + g(h_{t-1}, h_{t-1}; \theta)$ 作为 RNN 状态更新公式, 梯度爆炸原因? 解决?

解: 原因: 见上页, 略.

方法: 改用 Gated RNN, 如 LSTM.

互相矛盾，平衡二者？

§7 网络优化与正则化

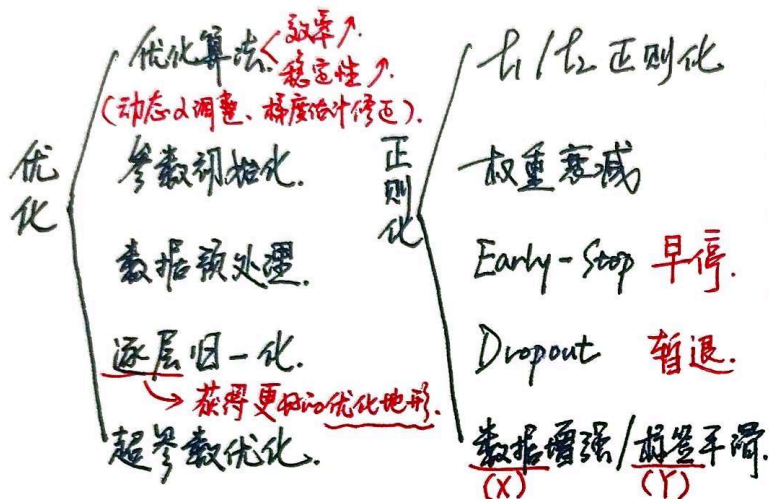
optimization Regularization

优化的需求：“大、多、高”（详见§4）。

目的：经验风险最小化！ $R(W, b)$ （详见§4）。

(train) 优化问题：Loss非凸；参数巨多；梯度消失/爆炸。

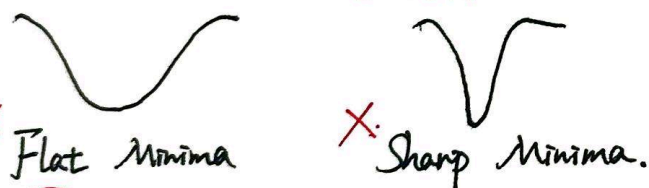
(test) 泛化问题：复杂度高，易过拟合；需正则化双进双能。



高维变量非凸优化的难点：逃离鞍点!! ^{Saddle Point}

梯度下降会在鞍点附近趋于停滞，很难run走……

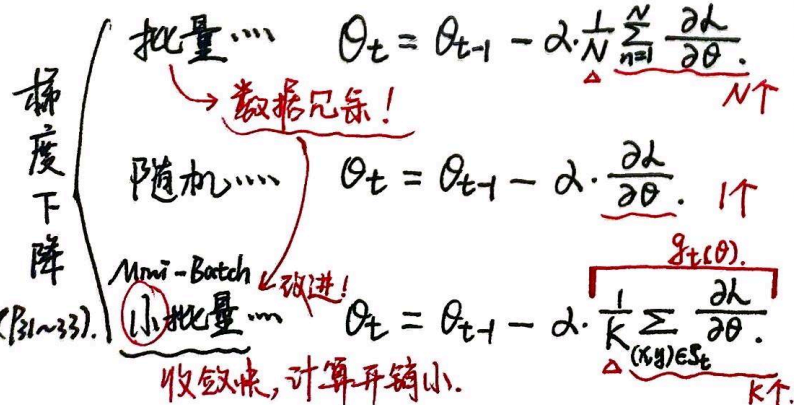
→ 在梯度方向上引入随机性，能否run走？



平坦最小值 尖锐最小值。

收敛至此，鲁棒性更好!!

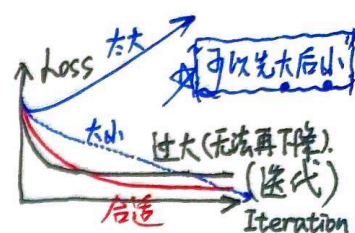
规模大，不必找全局Minima（反而会过拟合）。



Mini-Batch 梯度下降影响因素：

样本数量 K ，梯度估计，学习率 $\alpha > 0$ 。

对于学习率 α 的取值：



对于样本数量 K (Batch Size):

Batch Size $\uparrow$ ，方差 $\downarrow$ ，噪声 $\downarrow$ ，稳定，可取大 α 。

[习题7-1] 为什么 α 与Batch Size要成正比？

→ 线性缩放规则!

$$K \uparrow: -\alpha \cdot \frac{1}{K} \left(\frac{\partial L}{\partial \theta} + \dots + \frac{\partial L_K}{\partial \theta} \right)$$

$$mK \uparrow: -\alpha' \cdot \frac{1}{mK} \left(\frac{\partial L}{\partial \theta} + \dots + \frac{\partial L_{mK}}{\partial \theta} \right)$$

此时取 $\alpha' = m\alpha$ ，与分母的 m 相抵消。

即取 $\alpha \propto K$ 。

优化算法汇总：



2. 梯度估计修正

★ 动量法 $\rightarrow \Delta \theta_t = \rho \Delta \theta_{t-1} - \alpha g_t$

Nesterov 加速梯度

梯度截断。抑制·爆炸!!

3. 综合方法: Adam $\approx$ 动量法 + RMSprop.

↓ 再引入 Nesterov?
★ Nadam 算法。

§7.3

参数初始化

预训练...

随机...

固定值...

$(x \in [-r, r], r = \sqrt{3\sigma^2})$
 $\sigma^2 = \text{var}(x) = \frac{(b-a)^2}{12}$
 基于固定方差的...
 高斯分布/均匀分布.
 基于方差缩放的...
 Xavier / He.
 正交...
 经验!
 比如偏置 (Bias) 设为 0.

§7.4

算法在收敛全部/部分特征后不影响学习、预测

eg. 线性分类器 ← 尺度不变性.

eg. 最近邻分类器 ← 尺度敏感模型.

特征尺度不同, NN 训练难度 ↑

梯度下降效率 ↓. (并不指向最优).

(数据预处理)

归一化

Normalization

最大最小值归一化 $\hat{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$

标准化 (Z值) $\hat{x} = \frac{x - \mu}{\sigma}$

Whitening 降低冗余性. ($\sigma \neq 0$).
白化 eg. PCA.

§7.5

逐层归一化:

优点

更好的尺度不变性 → 缓解内部协变量偏移 (Input 变, 参数重新训)
 更平滑的优化地形. → 非饱和区, 避免梯度消失.

4种类型: (1) 批量归一化 (Batch Normalization) (BN)

(3) 权重...

(4) 局部响应...

(略)

(2) 层归一化 (Layer Normalization).

(用在 RNN, 可致缓解梯度消失/爆炸)

Batch N...

vs.

Layer N...

Batch			mean 均值	std 标准差
1	3	6	3	3
2	2	2	2	0
0	1	5	2	3
4	6	1	4	3
5	2	3	3	2
1	0	1	1	1
mean	2	3	3	
std	2	2	2	

对所有训练样本相同.
 均值方差是向量.
 对所有特征维度相同.
 均值方差是标量.

§7.6 超参数优化 Hyperparameter Optimization

超参数

网络结构: 连接, L, M, f.

优化参数: 方法, α , Batch Size.

正则化系数.

困难

优化难 — 组合优化.
 配置难 — 评估的时间代价高.

HO 方法: 网格搜索, 随机搜索.

贝叶斯优化, 动态资源分配.

神经架构搜索.

§7.7 正则化 → “损害优化”: 加约束/干扰过程!

限制 model 复杂度 → 避免过拟合 → 泛化能力 ↑

(1) L1/L2 正则化 $\theta^* = \arg \min_{\theta} \sum d + \lambda p(\theta)$

($\lambda \uparrow$, 约束能力 ↑, 过拟合 ↓).

正则化系数 λ
 $p=1$ 或 2 .

(2) 权重衰减. Weight Decay.

$\theta_t \leftarrow (1 - \beta) \theta_{t-1} - \alpha g_t$

(3) 早停. Early-Stop.

val 的误差率不再下降 ⇒ 停止迭代! (及时止损!!)



(4) 暂退. Dropout.

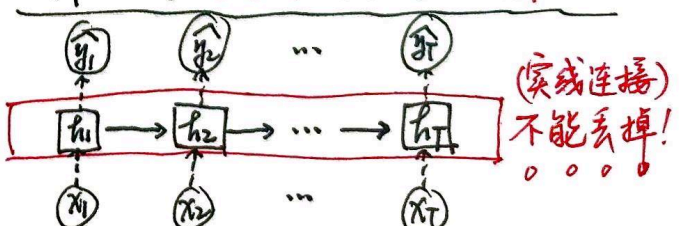
随机暂退一部分神经元 (及其连接边), 防止过拟合.

译作“暂退”比“丢弃”更贴切! — 邱xp.

[习题 7-10] 为什么不能在 RNN 的循环连接上直接 Dropout?

会损害 RNN 在时间维度上的记忆能力! (译四仰)

循环连接有误差累积效果, 会放大噪声!!



(5) 数据增强: 旋/翻/缩放/平移/加噪声.

(6) 标签平滑: $y = [0, 0, \dots, 1, \dots, 0, 0]$ 硬目标.

补: 知识蒸馏? 教师 Net → 学生 Net.

Attention Mechanism

§8 注意力机制与外部记忆.

§8.1 聚焦式 (有意识) $\uparrow$ 显著性 $\rightarrow$ e.g. 最大汇聚. 门控机制. $\downarrow$ (无意识)

§8.2 注意力机制步骤:

(1) 计算注意力分布 α : 输入 $\xleftrightarrow{\text{相关性?}}$ 查询向量 q .

"软性": $\alpha_n = p(z=n|X, q)$ 打分函数.

$$= \text{softmax}(s(x_n, q))$$

$$= \frac{\exp(s(x_n, q))}{\sum_{j=1}^N \exp(s(x_j, q))}$$

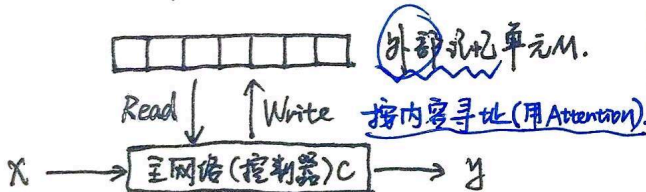
(2) 加权平均.

"软性": $\text{att}(X, q) = \sum_{n=1}^N \alpha_n x_n = \mathbb{E}_{z \sim p(z|X, q)}[x_z]$

变体: 硬性..., 键值对...

§8.3 自注意力模型: 增加层数; 权重 d_{ij} 动态生成.

§8.5 Memory Augmented 使用 QKV (查询-键-值) 模式. 记忆增强神经网络 (MANN) 简称: 记忆网络.



参数与记忆容量分离!

① 寻址 $\alpha_n = \text{softmax}(s(m_n, q))$ ② 读取 $r = \sum_{n=1}^N \alpha_n m_n$

举例: (1) 端到端记忆网络 (MemN2N).

(2) 神经图灵机 (NTM)

[习题8-5] 分析二者对外部记忆操作的异同点.

Unsupervised $\rightarrow$ 无Label.

§9 无监督学习.

§9.1 无监督特征学习. (PCA 稀疏编码. 自编码器. Auto-Encoder.)

§9.2 (概率) 密度估计. $\left\{ \begin{array}{l} \text{参数} \dots: \text{假设!} \\ \text{非参数} \dots: \text{分区域近似.} \end{array} \right.$

聚类 (Clustering). $\left\{ \begin{array}{l} \text{固定区域大小} V \\ \text{改变} V \text{ 控制并控制} K \end{array} \right.$

直方图方法; 核" K 近邻.

§10 模型独立的学习方式.

$\downarrow$ 不限于具体的模型!

§10.1 Ensemble Learning

集成学习: 集成多个模型, 群体决策!

How? $\left\{ \begin{array}{l} \text{直接平均(投票): } F(x) = \frac{1}{M} \sum_{m=1}^M f_m(x). \\ \text{加权平均.} \end{array} \right.$

(1) Bagging 类方法 $\left\{ \begin{array}{l} \text{Bagging} \\ \text{Random Forest 随机森林} \end{array} \right.$ (2) Boosting 类. e.g. AdaBoost.

§10.2 Semi-Supervised 半监督学习 $\left\{ \begin{array}{l} \text{自训练} \\ \text{协同训练} \end{array} \right.$ (少量Label, 大量无) $\rightarrow$ 2个不同视图的分类器 利用数据的互补性.

§10.3 Multi-Task 多任务学习 (归纳迁移学习). $\rightarrow$ 看成 相关任务 $\rightarrow$ 存在共享知识. now 为主. (表示(特征)参数、算法等).

共享机制: 硬共享, 软..., 层次..., 共享-私有.

§10.4 Transfer 迁移学习 $\left\{ \begin{array}{l} \text{归纳} \dots \\ \text{转导} \dots \end{array} \right.$

§10.5 Lifelong 终身学习 / Continuous 持续学习

§10.6 Meta-Learning 元学习 (学习的学习) $\left\{ \begin{array}{l} \text{基于优化器的} \dots \\ \text{模型无关的} \dots \end{array} \right.$